

UNIT 5

UNIT- V Arduino in IoT

Basics of Arduino: Introduction to Arduino – Types of Arduino – Arduino Toolchain – Arduino Programming Structure – Sketches – Pins -Input/Output From Pins Using Sketches – Introduction to Arduino Shields – Integration of Sensors and Actuators with Arduino-Connecting LEDs with Arduino, Connecting LCD with Arduino – Tinkercad Arduino simulation.

Introduction to Arduino

Arduino is a software as well as hardware platform that helps in making electronic projects. It is an open source platform and has a variety of controllers and microprocessors. There are various types of Arduino boards used for various purposes.

The Arduino is a single circuit board, which consists of different interfaces or parts. The board consists of the set of digital and analog pins that are used to connect various devices and components, which we want to use for the functioning of the electronic devices. Most of the Arduino consists of 14 digital I/O pins. The analog pins in Arduino are mostly useful for fine-grained control. The pins in the Arduino board are arranged in a specific pattern. The other devices on the Arduino board are USB port, small components (voltage regulator or oscillator), microcontroller, power connector, etc.. Arduino also simplifies the process of working with microcontrollers.

Features of Arduino:

- **Inexpensive** - Arduino boards are relatively inexpensive compared to other microcontroller platforms. The least expensive version of the Arduino module can be assembled by hand, and even the pre-assembled Arduino modules cost less than \$50
- **Cross-platform** - The Arduino Software (IDE) runs on Windows, Macintosh OSX, and Linux operating systems. Most microcontroller systems are limited to Windows.
- **Simple, clear programming environment** - The Arduino Software (IDE) is easy-to-use for beginners, yet flexible enough for advanced users to take advantage of as well. For teachers, it's conveniently based on the Processing programming environment, so students learning to program in that environment will be familiar with how the Arduino IDE works.
- **Open source and extensible software** - The Arduino software is published as open source tools, available for extension by experienced programmers. The language can be expanded through C++ libraries, and people wanting to understand the technical details can make the leap from Arduino to the AVR C programming language on which it's based. Similarly, you can add AVR-C code directly into your Arduino programs if you want to.
- **Open source and extensible hardware** - The plans of the Arduino boards are published under a Creative Commons license, so experienced circuit designers can make their own version of the module, extending it and improving it. Even relatively inexperienced users can build the breadboard version of the module in order to understand how it works and save money.

History of Arduino:

The project began in the Interaction Design Institute in Ivrea, Italy. Under the supervision of Casey Reas and Massimo Banzi, the Hernando Bar in 2003 created the Wiring (a development platform). It was considered as the master thesis project at IDII. The Wiring platform includes the PCB (Printed Circuit Board). The PCB is operated with the ATmega168 Microcontroller. The ATmega168 Microcontroller was an IDE. It was based on the library and processing functions, which are used to easily program the microcontroller.

In 2005, Massimo Banzi, David Cuartielles, David Mellis, and another IDII student supported the ATmega168 to the Wiring platform. They further named the project as Arduino.

The project of Arduino was started in 2005 for students in Ivrea, Italy. It aimed to provide an easy and low-cost method for hobbyists and professionals to interact with the environment using the actuators and the sensors. The beginner devices were simple motion detectors, robots, and thermostats.

In mid-2011, the estimated production of Arduino commercially was 300,000. In 2013, the Arduino boards in use were about 700,000.

Around April 2017, Massimo Banzi introduced the foundation of Arduino as the "new beginning for Arduino". In July 2017, Musto continued to pull many Open Source licenses and the code from the websites of the Arduino. In October 2017, Arduino introduced its collaboration with the ARM Holdings. The Arduino continues to work with architectures and technology vendors.

Hardware and Software

Arduino boards are generally based on microcontrollers from Atmel Corporation like 8, 16 or 32 bit AVR architecture based microcontrollers. The important feature of the Arduino boards is the standard connectors. Using these connectors, we can connect the Arduino board to other devices like LEDs or add-on modules called Shields.

The Arduino boards also consists of on board voltage regulator and crystal oscillator. They also consist of USB to serial adapter using which the Arduino board can be programmed using USB connection.

In order to program the Arduino board, we need to use IDE provided by Arduino. The Arduino IDE is based on Processing programming language and supports C and C++.

Types of Arduino

Arduino makes several different boards, each with different capabilities. In addition, part of being open source hardware means that others can modify and produce derivatives of Arduino boards that provide even more form factors and functionality.

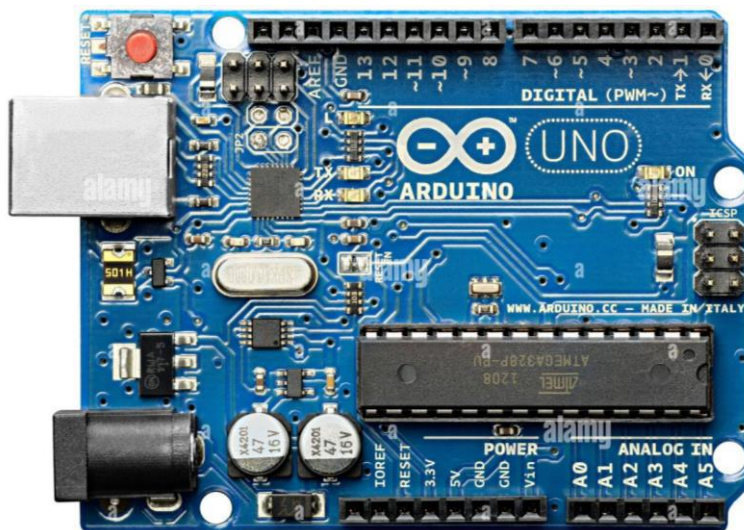
The Arduino boards are provided as open source that helps the user to build their projects and instruments according to their need. This electronic platform contains microcontrollers,

connections, LEDs and many more. There are various types of Arduino boards present in the market that includes Arduino UNO, Red Board, LilyPad Arduino, Arduino Mega, Arduino Leonardo. All these Arduino boards are different in specifications, features and uses and are used in different type of electronics project.

Entry-Level Arduino Boards

Arduino's entry-level category contains the microcontroller boards that most DIYers choose to use for their projects, as they offer straightforward features and come with heaps of documentation. This also means that they can lack the niche features that come with enhanced and IoT Arduino boards.

1.Arduino UNO



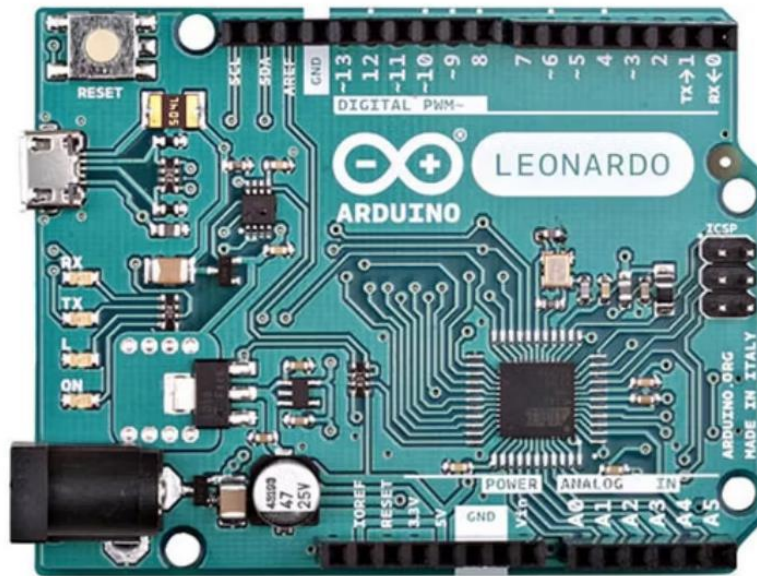
The development of Arduino UNO board is considered as new compared to other Arduino boards. This board comes up with numerous features that helps the user to use this in their project. The Arduino UNO uses the Atmega16U2 microcontroller that helps to increase the transfer rate and contain large memory compared to other boards. No extra devices are needed for the Arduino UNO board like joystick, mouse, keyboard and many more. The Arduino UNO contain SCL and SDA pins and also have two additional pins fit near to RESET pin.

The board contains 14 digital input pins and output pins in which 6 pins are used as PWM, 6 pins as analog inputs, USB connection, reset button and one power jack. The Arduino UNO board can be attached to computer system buy USB port and also get power supply to board from computer system. The Arduino UNO contains flash memory of size 32 KB that is used to the data in it. The other feature of the Arduino UNO is compatibility with other shield and can be combined with other Arduino products.

Basic Specs:

- Microcontroller: ATmega328P
- Memory: 2kB SRAM, 32kB flash, and 1kB EEPROM
- Communication: UART, IC2, and SPI
- Special Features: Replaceable chip

2.Arduino Leonardo



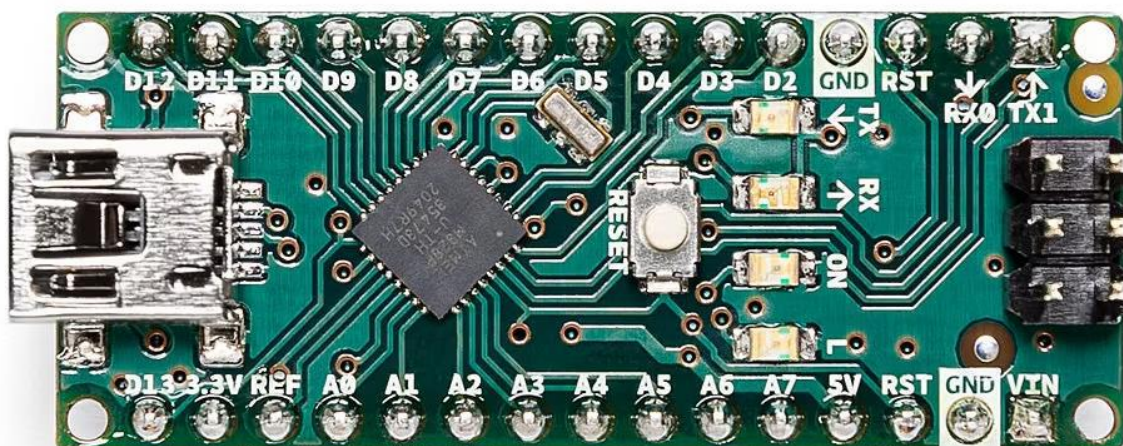
The Arduino Leonardo comes with essentially the same specifications as an Uno R3, only it features a micro-USB connector, has 20 digital and 17 analog pins, and has the ability to be used as a human interface device thanks to the ATmega32U4 chip that powers it. This means that your Leonardo can be used as a keyboard or mouse with a USB cable attached.

This type of Arduino is ideal for simple projects that need to interact with machines like computers, providing a huge range of different ideas to try for yourself.

Basic Specs:

- Microcontroller: ATmega32U4
- Memory: 2.5kB SRAM, 32kB flash, and 1kB EEPROM
- Communication: UART, I2C, and SPI
- Special Features: HID connectivity

3.Arduino Nano / Nano Every



The Arduino Nano and Nano Every are the smallest microcontroller boards offered by the company. Both boards feature the same pin layout, with 14 digital pins and 8 analog pins, though the Nano Every has a beefier microcontroller chip and improved program memory. These boards both come with pre-soldered headers that make them ideal for use with breadboards, but they lack the power jack that comes on larger boards.

Their breadboard compatibility makes these small boards great for those who like to make circuits that change all the time, like school teachers and prototype makers.

Basic Specs:

- Microcontroller: ATmega32U4 (Nano); ATmega4809 (Nano Every)
- Memory: 2kB SRAM, 32kB flash, and 1kB EEPROM (Nano); 6kB SRAM, 48kB flash, and 256B EEPROM (Nano Every)
- Communication: UART, IC2, and SPI
- Special Features: Breadboard-compatible and extremely small

4.Arduino Micro



The Arduino Micro boasts very similar features to the Leonardo, only the board is much smaller and only features 12 analog pins alongside its 20 digital ones. At only 18mm wide and 48mm long, this board is one of the smallest Arduino has ever made, making it ideal for creating a keyboard, mouse, and or other HID devices that need to be tiny.

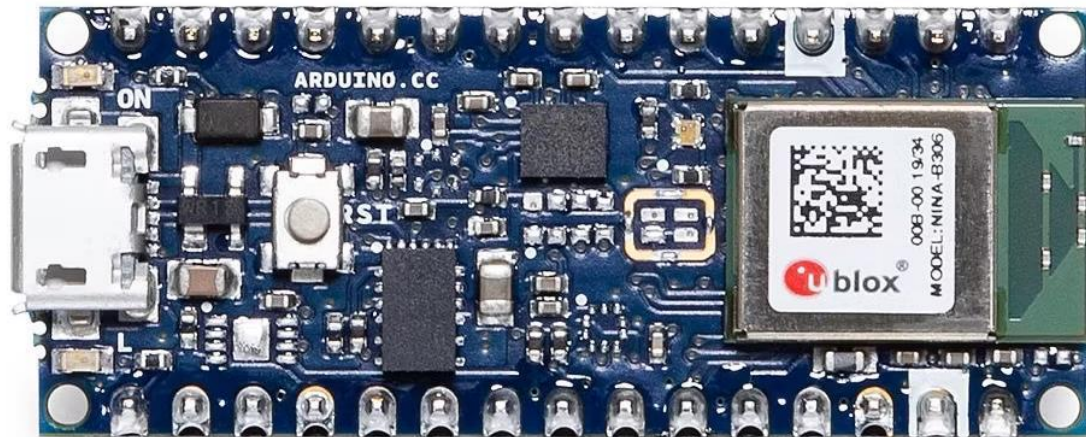
Basic Specs:

- Microcontroller: ATmega32U4
- Memory: 2.5kB SRAM, 32kB flash, and 1kB EEPROM
- Communication: UART, IC2, and SPI
- Special Features: HID connectivity and small form-factor

Enhanced Arduino Boards

Enhanced Arduino boards offer the features required to complete projects with greater complexity, while also providing improved performance for DIYers looking to push the limits.

1.Arduino Nano 33 BLE / Nano 33 BLE Sense



The Nano 33 BLE / Nano 33 BLE Sense is designed as an improved version of the Arduino Nano / Nano Every, featuring the same pin layout to make it nice and easy for DIYers. Both boards have a 32-bit Arm Cortex-M4 CPU running at 64MHz built into their nRF52840 chips, with 1MB of flash memory and 256kB of SRAM, making these boards incredibly powerful despite their small size.

They only come with 14 digital pins, but are packed with a host of sensors that don't come with regular Nanos. This sensor array includes an accelerometer, a gyroscope, and a magnetometer with 3-axis resolution, and the board comes with Bluetooth Low Energy (BLE) that makes it easy to transmit the data it collects.

Alongside all of these great features, the Nano 33 BLE Sense is also able to run edge computing applications using machine learning models from TensorFlow Lite.

Basic Specs:

- Microcontroller: nRF52840
- Memory: 256kB SRAM and 1MB flash
- Communication: UART, IC2, and SPI
- Special Features: Sensors, Bluetooth, and AI (Sense only)

2.Arduino MKR Zero

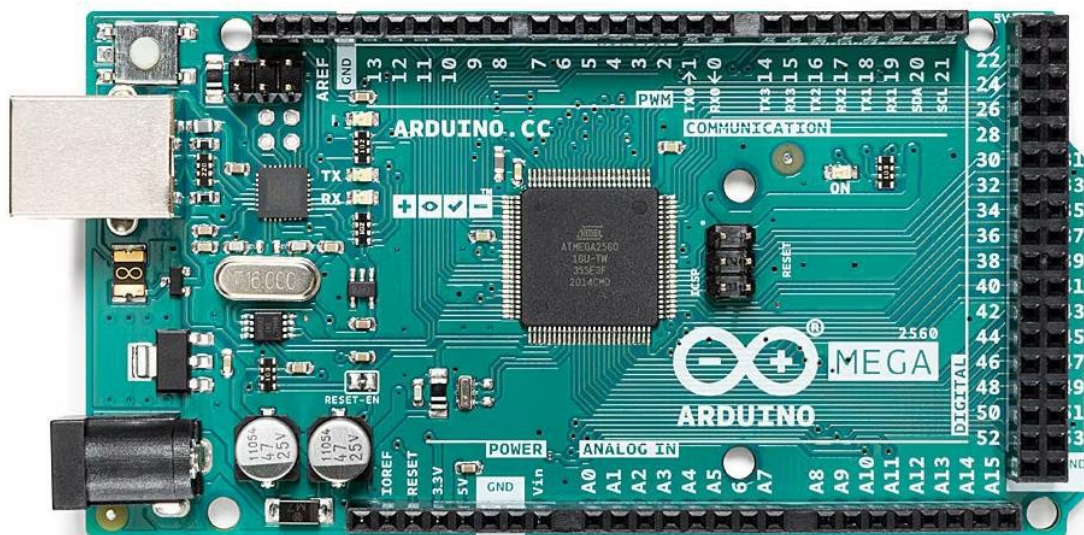


The Arduino MKR Zero is designed with music-making and other complex projects in mind, featuring a powerful Arm Cortex-M0 32-bit SAMD21 processor, native battery support, and a built-in microSD card reader. The board comes with 8 digital pins, 7 analog input pins, and 1 analog output pin. Thanks to the expandable storage that comes with this board, it is an excellent option for those working with a lot of code and a range of hardware components.

Basic Specs:

- Microcontroller: Arm Cortex-M0 32-bit SAMD21
- Memory: 32kB SRAM and 256kB flash
- Communication: UART, IC2, and SPI
- Special Features: Built-in battery connector, microSD card reader, powerful hardware

3.Arduino Mega 2560 R3



The Arduino Mega 2560 is similar to an Arduino Uno, only it features 54 digital pins, 16 analog pins, and 4 serial ports, along with being much larger and more powerful than the standard Uno. This board is great for DIYers in need of more pins, memory, or processing power without sacrificing the functionality that comes with regular Arduinos.

Basic Specs:

- Microcontroller: ATmega2560
- Memory: 8kB SRAM, 256kB flash, and 4kB EEPROM
- Communication: UART, IC2, and SPI
- Special Features: Large form-factor and serial ports

Arduino Toolchain

A toolchain is a set of software development tools used simultaneously to complete complex software development tasks or to deliver a software product. Each tool in the chain is itself a piece of software that serves a different function and is optimized to work together with other tools in the chain. Example programs in a toolchain are an assembler, linker and debugger.

Companies can customize toolchains to suit their software needs. Some software vendors also offer their own toolchains. One of the main benefits of using a toolchain instead of a disparate set of tools is the consistent programming environment throughout the development process and the smooth transition between tools as development progresses.

Example:

In programming, a toolchain is just a series of programming tools that work together in order to complete a task.

If we were cooking something, like carrot soup, the toolchain might include a carrot peeler, a produce knife, a pot, some water, and a stove.



Together, these tools help you create the desired output, carrot soup.

When we're developing programs for Arduino, we'll also be using a toolchain. This all happens behind the scenes. On our end, we literally only have to press a button for the code to get to the board, but it wouldn't happen without the toolchain.

The Arduino has a similar toolchain. When we start writing our code, and we become the author. We do this in the Arduino IDE, which is akin to a text editor. We also write the code in a programming language called C++, with a file type extension of .ino

The code that we write is called human readable code since it's meant for us to read and, hopefully, understand. However, the Arduino hardware doesn't understand C++ code.

It needs what's called machine code. In order to generate that machine code, we use a software tool called a compiler. Remember that Verify button in the Arduino IDE that looks like the checkmark?



When you press this button, it compiles the code using compiler software called AVR-GCC. This compiler software does a bunch of stuff. The main thing it does it rearrange some code and check for errors. This is like the professional editor at the publishing company. The compiler also translates the human readable code into machine code. There are some in-between file types and other fuzzy things that go on, but the final output of the compiler is a machine language saved as a .hex file.

In order to get this .hex file loaded onto our Arduino's integrated circuit, we need to press the Upload button.



This begins another piece of software called AVRDUDE, which sends the file to the integrated circuit.

Normally, we would have to use some external hardware to load that circuit. However, in Arduino's infinite wisdom, they loaded a small piece of software onto the integrated circuit that automatically comes with an Arduino board.

That piece of software is called a bootloader. It works with AVRDUDE to take the outputted .hex file and put it onto the flash memory of that Arduino's integrated circuit using only the USB cable.

Again, all we had to do was press the Upload button. This whole process happens behind the scenes.

Now the process is complete

What is the purpose of a toolchain?

The purpose of a toolchain is to have a group of linked software tools that are optimized for a specific programming process. The output generated by one tool in the chain is used by the next tool as the input.

By integrating tools together, the toolchain considers the various tool dependencies and, as a result, streamlines the software development process. For example, a programming language such as C++ may require language-specific tools, like a debugger or compiler. A development team that knows C++ can integrate one of these tools into their chain to avoid bottlenecks in the development process.

What should a toolchain include?

Toolchains may vary depending on the team using them and the product being delivered. However, basic toolchains often include the following components:

Assembler -- converts assembly language into code.

Linker -- links a set of program files together into a single program.

Debugger -- used to test and debug programs.

Compiler -- used to generate executable code from the source code of a program.

Runtime libraries -- used to interface with an operating system (OS). It enables the program to reference external functions and resources. An API is an example of this.

Developers will often write a script that links these tools together and automates some of the process. Complex software products will likely require more tools than this, but this basic assortment is generally involved in any software toolchain in some form, regardless of the product being delivered.

Some coding languages and platforms offer their own toolchains and enable customization within the integrated development environment platform. Apple's Xcode is one example of this.

Benefits of using toolchains

The main benefit of using a software toolchain is that it streamlines the development lifecycle. To a degree, the software development process can be automated, which is especially useful for DevOps implementations. In DevOps implementations, teams often employ a continuous delivery strategy and need all tools working simultaneously to be as efficient as possible.

Additionally, using a pre-built toolchain from a third party allows development teams to bypass the step of building a toolchain from scratch, which can be difficult and time consuming. Developers can use these preexisting toolchains and then customize them to the specific use case.

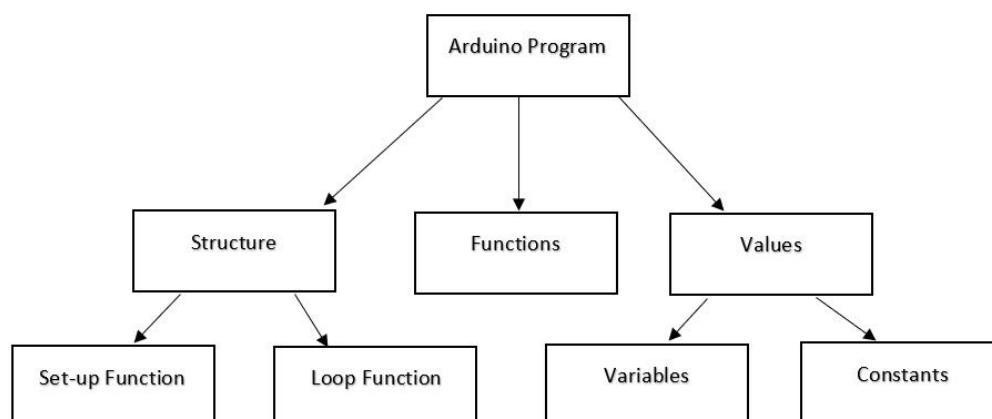
Using toolchains can also help expedite handoffs between teams that use different software tools by increasing visibility, security and productivity.

Arduino Programming Structure

Sketch – The first new terminology is the Arduino program called “sketch”, Different languages can be used to execute different functions by using electronic machines. These languages aid in giving commands to the machine. There are a lot of different programming languages, and each language has its own commands, syntax, and structure of writing a program. The language used for Arduino is C++. The Arduino program structure is briefly explained in this discourse.

Arduino Programing Overview

The Arduino program is divided into three main parts that are structure, values, and functions.



1. Structure

Structure consist of two main functions –

Setup() function:

The setup() function is called when a sketch starts. Use it to initialize variables, pin modes, start using libraries, etc. The setup() function will only run once, after each powerup or reset of the Arduino board.

Eg:

```
1 int buttonPin = 3;
2
3 void setup() {
4   // put your setup code here, to executed once:
5   Serial.begin(9600);
6   pinMode(buttonPin, INPUT);
7   Serial.println("This is setup code");
8 }
9
10 void loop() {
11   // ...
12 }
```

Output:



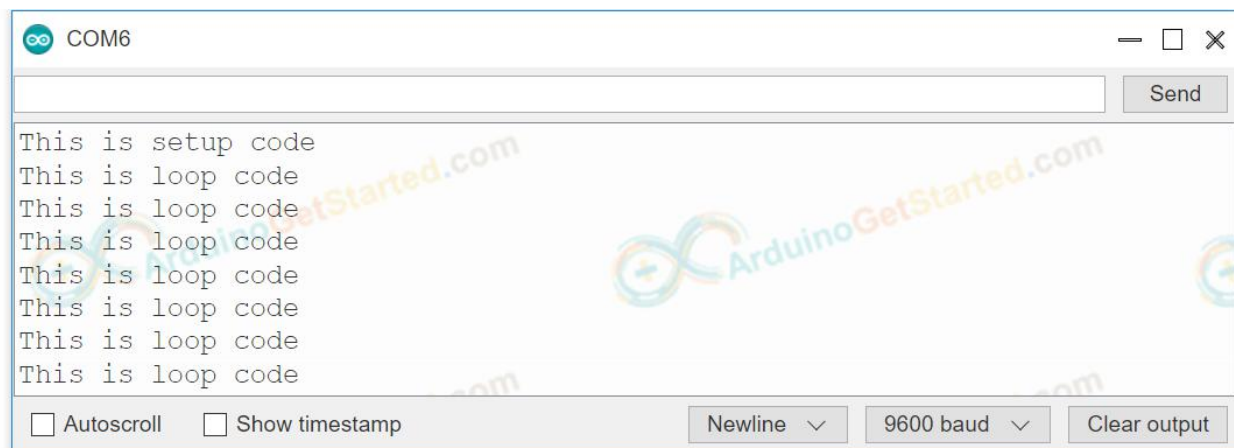
Loop() function:

After creating a setup() function, which initializes and sets the initial values, the loop() function does precisely what its name suggests, and loops consecutively, allowing your program to change and respond. Use it to actively control the Arduino board.

Eg:

```
1 void setup() {
2   // put your setup code here, to executed once:
3   Serial.begin(9600);
4   Serial.println("This is setup code");
5 }
6
7 void loop() {
8   // put your main code here, to run repeatedly:
9   Serial.println("This is loop code");
10  delay(1000);
11 }
```

Output:



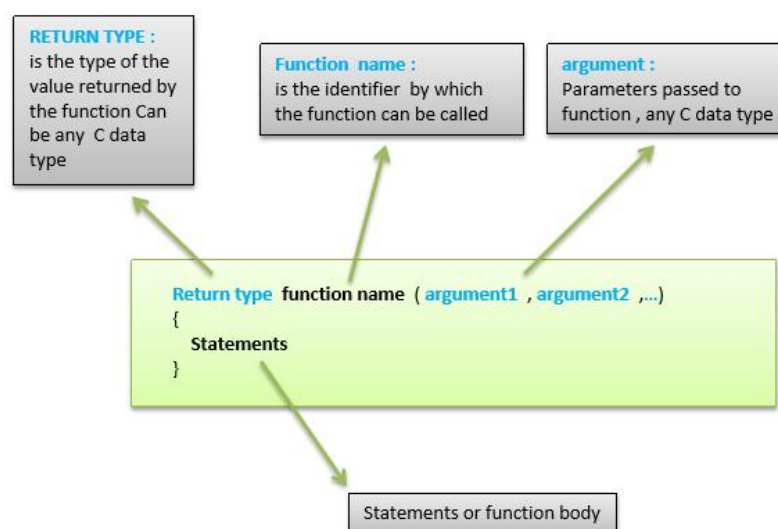
```
This is setup code
This is loop code
This is loop code
This is loop code
This is loop code
This is loop code
This is loop code
This is loop code
```

2.Functions

Functions allow structuring the programs in segments of code to perform individual tasks. The typical case for creating a function is when one needs to perform the same action multiple times in a program.

Standardizing code fragments into functions has several advantages –

- Functions help the programmer stay organized. Often this helps to conceptualize the program.
- Functions codify one action in one place so that the function only has to be thought about and debugged once.
- This also reduces chances for errors in modification, if the code needs to be changed.
- Functions make the whole sketch smaller and more compact because sections of code are reused many times.
- They make it easier to reuse code in other programs by making it modular, and using functions often makes the code more readable.



Function Declaration

A function is declared outside any other functions, above or below the loop function.

We can declare the function in two different ways –

The first way is just writing the part of the function called **a function prototype** above the loop function, which consists of –

- Function return type
- Function name
- Function argument type, no need to write the argument name

Function prototype must be followed by a semicolon (;).

The following example shows the demonstration of the function declaration using the first method.

Example

```
int sum_func (int x, int y) // function declaration {  
    int z = 0;  
    z = x+y ;  
    return z; // return the value  
}  
  
void setup () {  
    Statements // group of statements  
}  
  
Void loop () {  
    int result = 0 ;  
    result = Sum_func (5,6) ; // function call  
}
```

3.Values

A value is a piece of data that can be manipulated by the program. This could be a number, a character, a string of text, or a logical value (such as "true" or "false"). In Arduino programs, values are often stored in variables, which are named containers that can hold a single value. These values can then be used in calculations, compared to other values, or used to control the behavior of the program. For example, a variable named "x" might be used to store the current temperature reading from a sensor, which could then be used to control a heating or cooling system.

Values Are 2 Types

1.Variable

2.Constant

1.Variable:

The variables are defined as the place to store the data and values. It consists of a name, value, and type.

The variables can belong to any data type such as int, float, char, etc.

Consider the below example:

```
int pin = 8;
```

2.Constant:

The constants in Arduino are defined as the predefined expressions. It makes the code easy to read. The constants in Arduino are defined as:

Logical level Constants:

The logical level constants are true or false.

The value of true and false are defined as 1 and 0. Any non-zero integer is determined as true in terms of Boolean language. The true and false constants are type in lowercase rather than uppercase (such as HIGH, LOW, etc.).

Pin level Constants:

The digital pins can take two value HIGH or LOW.

In Arduino, the pin is configured as INPUT or OUTPUT using the pinMode() function. The pin is further made HIGH or LOW using the digitalWrite() function.

Input/Output From Pins Using Sketches

In the Arduino platform, a sketch is a program written in the Arduino programming language. Sketches are used to control the behavior of the Arduino board, including reading input from and writing output to the board's pins.

Input and output can be performed using the digital and analog pins on the Arduino board. Digital pins can be used to read digital inputs, such as the state of a button, or to write digital outputs, such as turning on an LED. Analog pins, on the other hand, can be used to read analog inputs, such as the output of a sensor, or to write analog outputs, such as dimming an LED.

To read and write to the pins, the sketch must use the appropriate Arduino functions. For example, the *digitalRead()* function can be used to read the state of a digital input pin, while the *analogWrite()* function can be used to write an analog output to a pin. These functions are typically used within the setup() and loop() functions of the sketch, which are special functions that are called automatically when the sketch is executed.

Here is an example of a simple sketch that reads a digital input from pin 2 and writes the state of that input to an LED connected to pin 13:

```
void setup() {  
    // Set the LED pin as an output  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    // Read the state of the input pin  
    int inputState = digitalRead(2);  
  
    // Write the state of the input to the LED  
    digitalWrite(13, inputState);  
}
```

In this sketch, the `setup()` function is used to configure pin 13 as an output, and the `loop()` function is used to read the input from pin 2 and write it to the LED on pin 13. This sketch will continue to run indefinitely, reading the input and updating the LED every time the `loop()` function is called.

Introduction to Arduino Shields

Arduino shields:

Arduino shields are the boards, which are plugged over the Arduino board to expand its functionalities. There are different varieties of shields used for various tasks, such as Arduino motor shields, Arduino communication shields, etc.

Shield is defined as the hardware device that can be mounted over the board to increase the capabilities of the projects. It also makes our work easy. For example, Ethernet shields are used to connect the Arduino board to the Internet.

The pin position of the shields is similar to the Arduino boards. We can also connect the modules and sensors to the shields with the help of the connection cable.

Arduino motor shields help us to control the motors with the Arduino board.

Why do we need Shields?

The advantages of using Arduino shields are listed below:

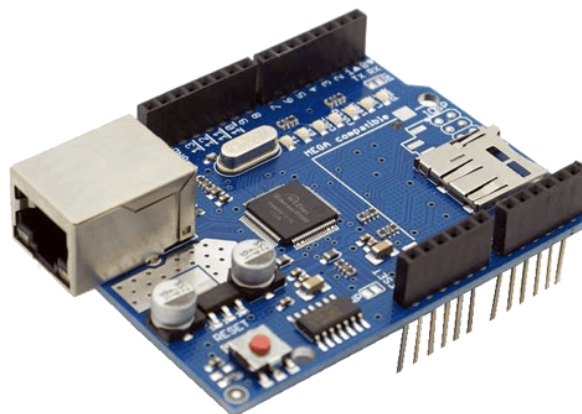
- It adds new functionalities to the Arduino projects.
- The shields can be attached and detached easily from the Arduino board. It does not require any complex wiring.
- It is easy to connect the shields by mounting them over the Arduino board.
- The hardware components on the shields can be easily implemented.

Types of Shields

The popular Arduino shields are listed below:

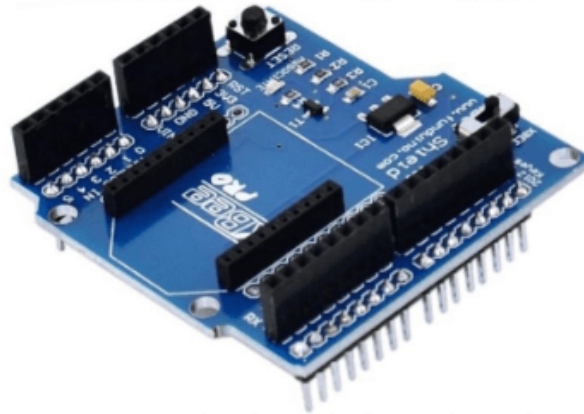
- **Ethernet shield**
- **Xbee Shield**
- **Proto shield**
- **Relay shield**
- **Motor shield**
- **LCD shield**
- **Bluetooth shield**
- **Capacitive Touchpad Shield**

1.Ethernet shield



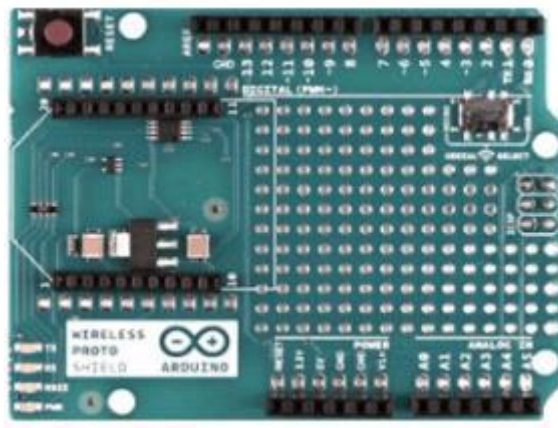
- The Ethernet shields are used to connect the Arduino board to the Internet. We need to mount the shield on the top of the specified Arduino board.
- The USB port will play the usual role to upload sketches on the board.
- The latest version of Ethernet shields consists of a micro SD card slot. The micro SD card slot can be interfaced with the help of the SD card library.
- We can also connect another shield on the top of the Ethernet shield. It means that we can also mount two shields on the top of the Arduino board.

2.Xbee Shield



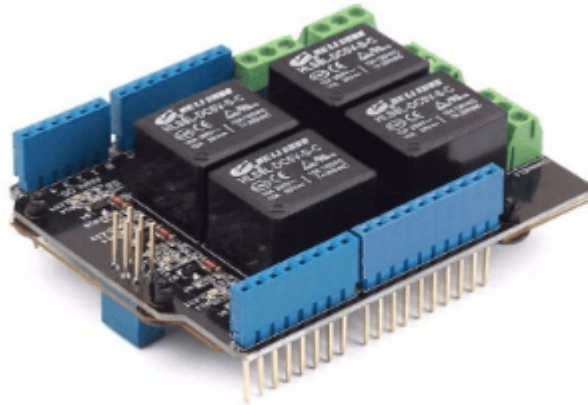
- We can communicate wirelessly with the Arduino board by using the Xbee Shield with Zigbee.
- It reduces the hassle of the cable, which makes Xbee a wireless communication model.
- The Xbee wireless module allows us to communicate outdoor upto 300 feet and indoor upto 100 feet.
- It can also be used with different models of Xbee.

3.Proto shield



- Proto shields are designed for custom circuits.
- We can solder electronic circuits directly on the shield.
- The shield consists of two LED pads, two power lines, and SPI signal pads.
- The IOREF (Input Output voltage REference) and GND (Ground) are the two power lines on the board.
- We can also solder the SMD (Surface Mount Device) ICs on the prototyping area. A maximum of 24 pins can be integrated onto the SMD area.

4. Relay shield



- The Arduino digital I/O pins cannot bear the high current due to its voltage and current limits. The relay shield is used to overcome such situation. It provides a solution for controlling the devices carrying high current and voltage.
- The shield consists of four relays and four LED indicators.
- It also provides NO/NC interfaces and a shield form factor for the simple connection to the Arduino board.
- The LED indicators depicts the ON/OFF condition of each relay.
- The relay used in the structure is of high quality.
- The NO (Normally Open), NC (Normally Closed), and COM pins are present on each relay.
- The applications of the Relay shield include remote control, etc.

5. Motor shield



- The motor shield helps us to control the motor using the Arduino board.
- It controls the direction and working speed of the motor. We can power the motor shield either by the external power supply through the input terminal or directly by the Arduino.
- We can also measure the absorption current of each motor with the help of the motor shield.
- The motor shield is based on the L298 chip that can drive a step motor or two DC motors. L298 chip is a full bridge IC. It also consists of the heat sinker, which increases the performance of the motor shield.

- It can drive inductive loads, such as solenoids, etc.
- The operating voltage is from 5V to 12V.
- The applications of the motor shield are intelligent vehicles, micro-robots, etc.

6.LCD shield



- The keypad of LCD (Liquid Crystal Display) shield includes five buttons called as up, down, left, right, and select.
- There are 6 push buttons present on the shield that can be used as a custom menu control panel.
- It consists of the 1602 white characters, which are displayed on the blue backlight LCD.
- The LED present on the board indicates the power ON.
- The five keys present on the board helps us to make the selection on menus and from board to our project.
- The LCD shield is popularly designed for the classic boards such as Duemilanove, UNO, etc.

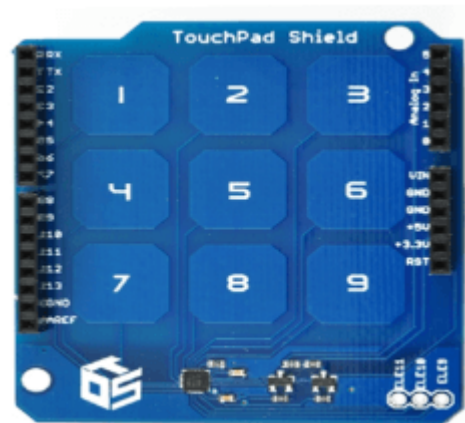
7.Bluetooth shield



- The Bluetooth shield can be used as a wireless module for transparent serial communication.
- It includes a serial Bluetooth module. D0 and D1 are the serial hardware ports in the Bluetooth shield, which can be used to communicate with the two serial ports (from D0 to D7) of the Arduino board.

- We can install Groves through the two serial ports of the Bluetooth shield called a Grove connector. One Grove connector is digital, while the other is analog.
- The communication distance of the Bluetooth shield is upto 10m at home without any obstacle in between.

8.Capacitive Touchpad shield



- It has a touchpad interface that allows to integrate the Arduino board with the touch shield.
- The Capacitive touchpad shield consists of 12 sensitive touch buttons, which includes 3 electrode connections and 9 capacitive touch pads.
- The board can work with the logic level of 3.3V or 5V.
- We can establish a connection to the Arduino project by touching the shield.

Integration of Sensors and Actuators with Arduino

Sensor

A sensor is a device that detects and responds to some type of input from the physical environment. The input can be light, heat, motion, moisture, pressure or any number of other environmental phenomena. The output is generally a signal that is converted to a human-readable display at the sensor location or transmitted electronically over a network for reading or further processing.

Sensors play a pivotal role in the internet of things (IoT). They make it possible to create an ecosystem for collecting and processing data about a specific environment so it can be monitored, managed and controlled more easily and efficiently. IoT sensors are used in homes, out in the field, in automobiles, on airplanes, in industrial settings and in other environments.

Actuators

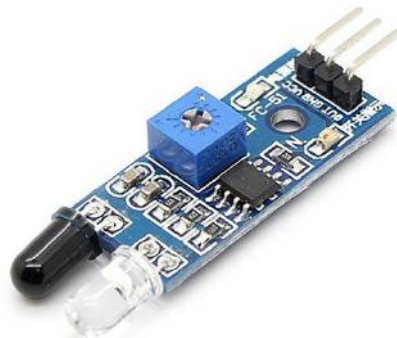
An actuator is a device that actuates something. In simple words, it produces motion. It helps other devices move and interact with their surrounding environment. The actuator takes in energy and uses it to help the device move. Servo motors and DC motors are types of electric actuators.

Integration of IR Sensor with Arduino

WHAT IS IR SENSOR?

An IR sensor is a device that measures the Infrared radiation in its surroundings and gives an electric signal as an output. An IR sensor can measure the heat of an object as well as can detect the motion of the objects.

IR technology is used in our day-to-day life and also in industries for different purposes. For example, TVs use an IR sensor to understand the signals which are transmitted from a remote control. The main benefits of IR sensors are low power usage, their simple design & their convenient features. IR signals are not noticeable by the human eye. The IR radiation in the electromagnetic spectrum can be found in the regions of the visible & microwave. Usually, the wavelengths of these waves range from $0.7\text{ }\mu\text{m}$ to $1000\mu\text{m}$. The IR spectrum can be divided into three regions near-infrared, mid, and far-infrared. The near IR region's wavelength ranges from $0.75 - 3\mu\text{m}$, the mid-infrared region's wavelength ranges from 3 to $6\mu\text{m}$ & the far IR region's infrared radiation's wavelength is higher than $6\mu\text{m}$.



As the Infrared Radiation is invisible to our eyes, it can be detected by the Infrared Sensor. An IR sensor is a photodiode that is sensitive to IR light. When IR light falls on the photodiode, the resistances and the output voltages will change in proportion to the magnitude of the IR light received.

TYPES OF IR SENSORS

1. Active IR Sensor.
2. Passive IR Sensor.

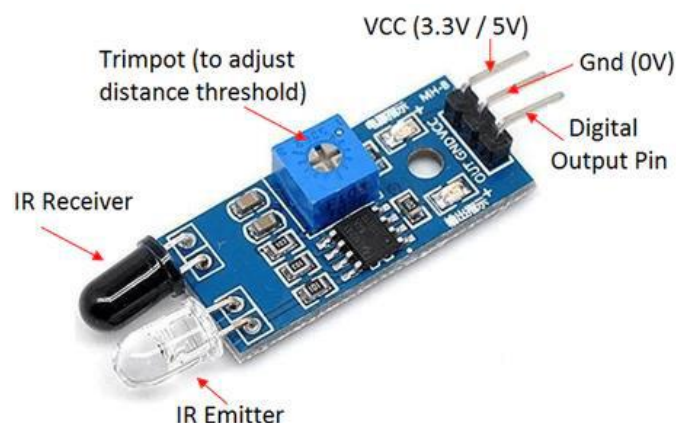
1. ACTIVE IR SENSOR

Active infrared sensors both emit and detect infrared radiation. Active IR sensors have two parts: a light-emitting diode (LED) as an Infrared Radiation transmitter and a Photodiode as an Infrared Radiation receiver. When an object comes close to the sensor, the infrared light from the LED reflects off of the object and is detected by the receiver. Active IR sensors act as proximity sensors, and they are commonly used in obstacle detection systems (such as in robots).

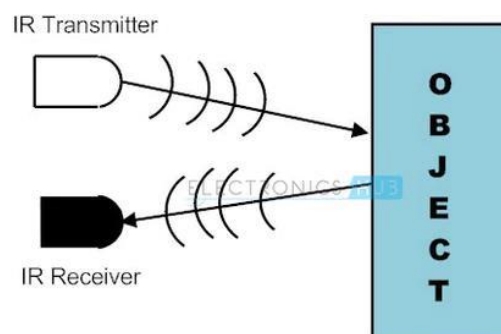
2. PASSIVE IR SENSOR

Passive infrared (PIR) sensors only detect infrared radiation and do not emit it from an LED. PIR sensors are most commonly used in motion-based detection, such as in-home security systems. When a moving object that generates infrared radiation enters the sensing range of the detector, the difference in IR levels between the two pyroelectric elements is measured. The sensor then sends an electronic signal to an embedded computer, which in turn triggers an alarm.

HOW DOES AN IR SENSOR MODULE WORK:



As we know till now that the active IR Sensors emit Infrared waves and as-well-as detect the Infrared waves. The IR sensor module exactly works on the same phenomenon. The IR Sensor module contains an Infrared LED and an infrared photodiode.

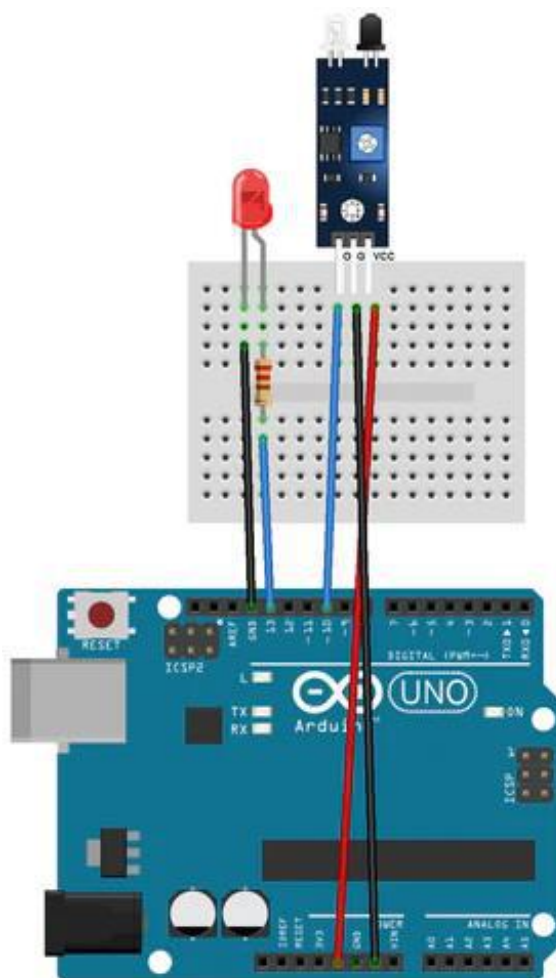


INTERFACING IR SENSOR MODULE WITH ARDUINO

HARDWARE REQUIRED

We need the following components:

1. Arduino
2. Breadboard
3. Jumper wires
4. IR Sensor module
5. LED
6. 220-ohm resistor



ARDUINO CODE FOR INTERFACING IR SENSOR MODULE WITH ARDUINO

```
int LEDpin = 13;
int obstaclePin = 10;
int hasObstacle = LOW; // LOW MEANS NO OBSTACLE

void setup() {
  pinMode(LEDpin, OUTPUT);
  pinMode(obstaclePin, INPUT);
  Serial.begin(9600);
}

void loop() {
  hasObstacle = digitalRead(obstaclePin);
  if (hasObstacle == HIGH) {
    Serial.println("Stop something is ahead!!");
    digitalWrite(LEDpin, HIGH);
  }
  else {
    Serial.println("Path is clear");
    digitalWrite(LEDpin, LOW);
  }
  delay(200);
}
```

UPLOADING THE CODE TO THE ARDUINO BOARD:

Step 1: Connect the Arduino board with the computer using a USB cable.

Step 2: Next type the code mentioned above.

Step 3: Select the right board and port.

Step 4: Upload the code to the Arduino.

Step 5: The LED will glow when any obstacle is detected by the IR sensor.

Integration of Actuator with Arduino

Integrating servo motor with Arduino Uno. Servo Motor is an electrical linear or rotary actuator which enables precise control of linear or angular position, acceleration or velocity. Usually servo motor is a simple motor controlled by a servo mechanism, which consists of a positional sensor and a control circuit. Servo motor is commonly used in applications like robotics, testing automation, manufacturing automation, CNC machine etc. The main characteristics of servo motor are low speed, medium torque, and accurate position.

Components Required

- Arduino Uno
- Servo Motor
- Jumper Wires

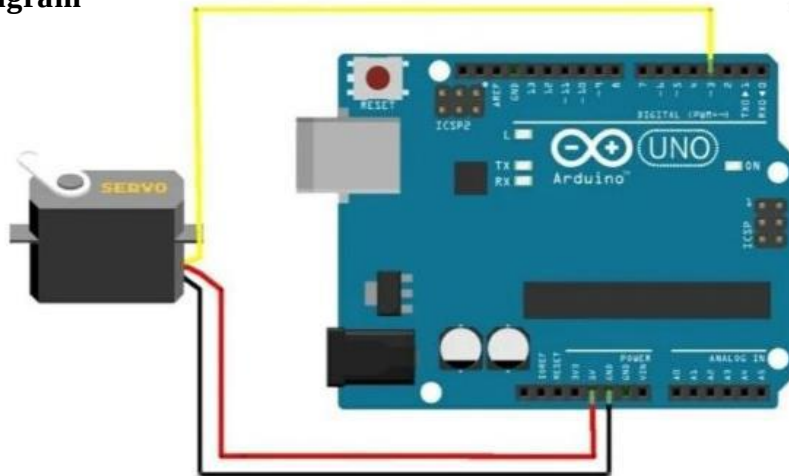
Hobby Servo Motor



Specifications

- Servo Motor consists of three pins: VCC, GROUND, and PWM signal pin.
- The VCC and GND is for providing power supply, the PWM input pin is used for controlling the position of the shaft by varying pulse width.
- Minimum Rotational Angle : 0 degrees
- Maximum Rotational Angle : 180 degrees
- Operating Voltage : +5V
- Torque : 2.5 kg/cm
- Operating Speed : 0.1 s/60 degree
- DC Supply Voltage : 4.8V to 6V

Circuit Diagram



Description

- The VCC pin (Red Color) of the Servo Motor is connected to the 5V output of the Arduino Board.
- The GND pin (Brown Color) of the Servo Motor is connected to the GND pin of the Arduino Board.
- The PWM input pin (Yellow Color) of the Servo Motor is connected to the PWM output pin of the Arduino board.

Working

The hobby servo motor which we use here consists of 4 different parts as below.

- Simple DC Motor
- Potentiometer
- Control Circuit Board
- Gear Assembly

Potentiometer is connected to output shaft of the servo motor helps to detect position. Based on the potentiometer value, the control circuit will rotate the motor to position the shaft to a certain angle. The position of the shaft can be controlled by varying the pulse width provided in the PWM input pin. Gear assembly is used to improve the torque of the motor by reducing speed.

Program

```
#include <Servo.h>

Servo servo;

int angle = 10;

void setup() {
    servo.attach(3);
    servo.write(angle);
}

void loop()
{
    for(angle = 10; angle < 180; angle++)
    {
        servo.write(angle);
        delay(15);
    }
    for(angle = 180; angle > 10; angle--)
    {
        servo.write(angle);
        delay(15);
    }
}
```

Code Explanation

- The position of the shaft is kept at 10 degrees by default and the Servo PWM input is connected to the 3rd pin of the Arduino Uno.
- In the first for loop, motor shaft is rotated from 10 degrees to 180 degrees step by step with a time delay of 15 milliseconds.
- Once it reaches 180 degree, it is programmed to rotate back to 10 degree step by step with a delay of 15 milliseconds in the second for loop.

Functioning

Wire the circuit properly as per the circuit diagram and upload the program. You can see that the servo motor is rotating as per the program.

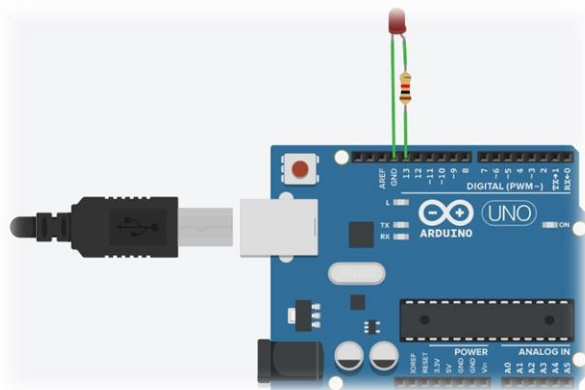
Connecting LEDs with Arduino

Light-emitting diode (LED) is a widely used standard source of light in electrical equipment. It has a wide range of applications ranging from your mobile phone to large advertising billboards. They mostly find applications in devices that show the time and display different types of data. In this Program we will Connect The LED and Blink it. Blinking means turning ON an LED for a few seconds, then OFF for seconds, repeatedly.

REQUIRED COMPONENTS

S.N.	COMPONENTS	QUANTITY
1.	Arduino	1
2.	LED	1
3.	Resistor (280 ohm)	1
4.	Connecting Wires	Few

CIRCUIT DIAGRAM



CIRCUIT EXPLANATION

Here, The anode (+) and cathode (-) terminals of LED are connected to pin 13 and ground (GND) of Arduino Uno respectively and a resistor of 280 ohm is placed between the LED anode terminal and pin number 13 which help us to limit the current and prevent LED from burning.

STEP 2: CODE

```
void setup()
{
  pinMode(13, OUTPUT);
```

```

}

void loop() {

digitalWrite(13, HIGH); delay(1000); digitalWrite(13, LOW); delay(1000);

}

```

Functioning

Wire the circuit properly as per the circuit diagram and upload the program. You can see that the the Blinking an LED as per the program.

Connecting LCD with Arduino

Display devices are used to visually display the information we are working with. LCD (Liquid Crystal Display) screens are one of many display devices that makers use. We have libraries to control specific LCD functions which make it ridiculously easy to get up and running with LCDs.

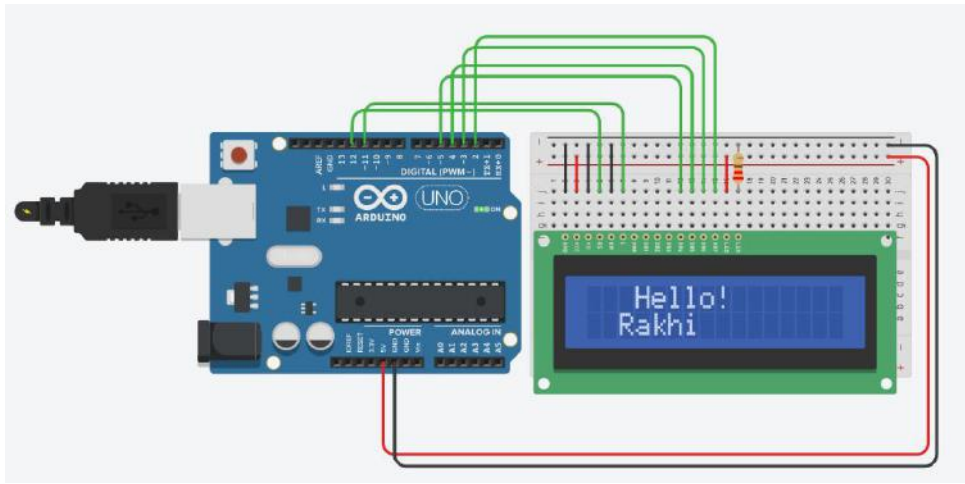
We are going to connect an LCD screen to our Arduino and make it print some stuff to the display, grab whatever you need and get started

- **Solderless Breadboard**
- **Arduino Board**
- **USB Cable**
- **Potentiometer (10k ohm)**
- **LCD Screen (16x2)**
- **Jumper Cables**

What is the LCD and how do it work?

A liquid crystal display is essentially a liquid crystal sandwiched between 2 pieces of glass, the liquid crystal reacts depending on current applied. Our LCD is a white on black, 16 by 2 character LCD that we will use to display symbols. Graphical LCDs also exist, but today we are just focusing on the character variety.

Each character is off by default and is a matrix of small dots of liquid crystal. These dots make up the numbers and letters that we display on screens. The actual coding that goes into making these characters appear is quite complicated, luckily for you, the people over at Arduino.cc have made a library of functions for the LCD that we can import into our sketch, The LCD also has a backlight and contrast control options. The backlight will shine through the pieces of glass the screen is made of to display the characters on the screen. The contrast will control how dark (or light) the characters appear. We will use a variable resistor to control contrast, and we will set the backlight to being on.



Circuit Explanation:

This circuit is the basic circuit for Arduino projects with LCD Display. If you know how to wire up a LCD Display with an Arduino then you can do any Arduino Projects including the LCD Display and the Arduino. You can use any Arduino flavor for this circuit, but just remember to put a current limiting resistor between the the pin 16 of the LCD Display to GND. Also if you want you can connect a 10k ohm Potentiometer to pin 3 of the LCD Display, remember you need to connect pin 1 of the POT to VCC - 5V and pin 2 of the POT to GND.

Code:

```
#include<LiquidCrystal.h>

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup()
{
  lcd.begin(16, 2);
}

void loop()
{
  lcd.setCursor(0,0);
  lcd.print(" Hello!");
  lcd.setCursor(2,1);
  lcd.print("Rakhi");
}
```

Functioning

Wire the circuit properly as per the circuit diagram and upload the program. You can see that the LCD Display With Some text as per the program.

Tinkercad Arduino simulation.

Arduino Simulator

The Arduino simulator is a **virtual portrayal of the circuits of Arduino** in the real world. We can create many projects using a simulator without the need for any hardware.

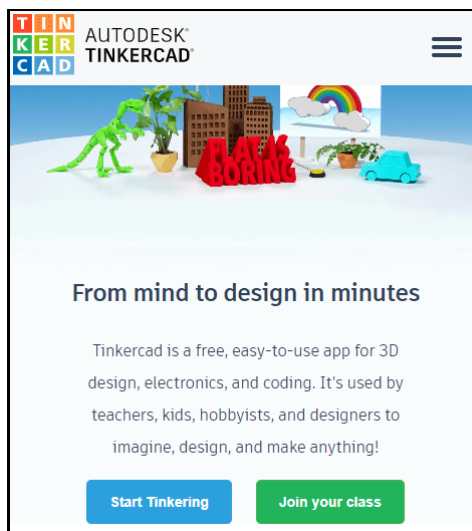
The Simulator helps beginner and professional designers to learn, program, and create their projects without wasting time on collecting hardware equipments.

we are using the **Autodesk Tinkercad Simulator**.

The steps to access the TINKERCAD are listed below:

1. Open the official website of tinkercad. : <https://www.tinkercad.com/>

A window will appear, as shown below:



2. Click on the three horizontal lines present on the upper right corner.
3. Click on the '**Sign in**' option, if you have an account in Autodesk. Otherwise, click on the '**JOIN NOW**' option if you don't have an account, as shown below:

The **SIGN IN** window will appear as:

We can select any sign-in method. Specify the username and password.

We already have an account in Autodesk, so we will sign-in directly with the username and password.

The **JOIN** window will appear as:

4. Now, a window will appear,
5. Click on the 'Create new circuit' option to start designing the Arduino circuit, as shown above.
6. The 'Circuits' option will also show the previous circuits created by user. The design option is used for creating the 3D design, which is of no use in Arduino

Autodesk Tinkercad Simulator

It is also a simulator that is used to design virtual circuits.

Features of Tinkercad

The features of Tinkercad are listed below:

- **Glow and move circuit assembly.** It means we can use the components of a circuit according to the project requirement. Glow here signifies the glowing of LED.
- **Integrated product design.** It means the electronic components used in the circuitry are real.
- **Arduino Programming.** We can directly write the program or code in the editor of the simulator.
- We can also consider some **ready-made examples** provided by the tinkercad for better understanding.
- **Realtime simulation.** We can prototype our designs within the browser before implementing them in real-time.

Advantages of using Simulator

There are various advantages of using simulator, which are listed below:

- It saves money, because there is no need to buy hardware equipments to make a project.
- The task to create and learn Arduino is easy for beginners.
- We need not to worry about the damage of board and related equipments.
- No messy wire structure required.
- It helps students to eliminate their mistakes and errors using simulator.
- It supports line to line debugging, and helps to find out the errors easily.
- We can learn the code and build projects anywhere with our computer and internet connection.
- We can also share our design with others.

SSSS